Parallel Seam Carving using OpenMP

Assignment 1 — High Performance Computing

Plabon Shaha and Guillem Masdemont
University of Ljubljana, Spring Semester, EMAI Cohort 25–27

March 24, 2026

Exercise 1: Execution times are measured for the five resolutions using 1, 2, 4, 8, 16, and 32 threads. We remove 128 seams and average timings across 5 experiments. Performance is evaluated via speed-up, defined as $S = \frac{t_s}{t_p}$, where t_s is the sequential runtime and t_p is the parallel runtime. The results are presented in figure 1 for a 4 core test:

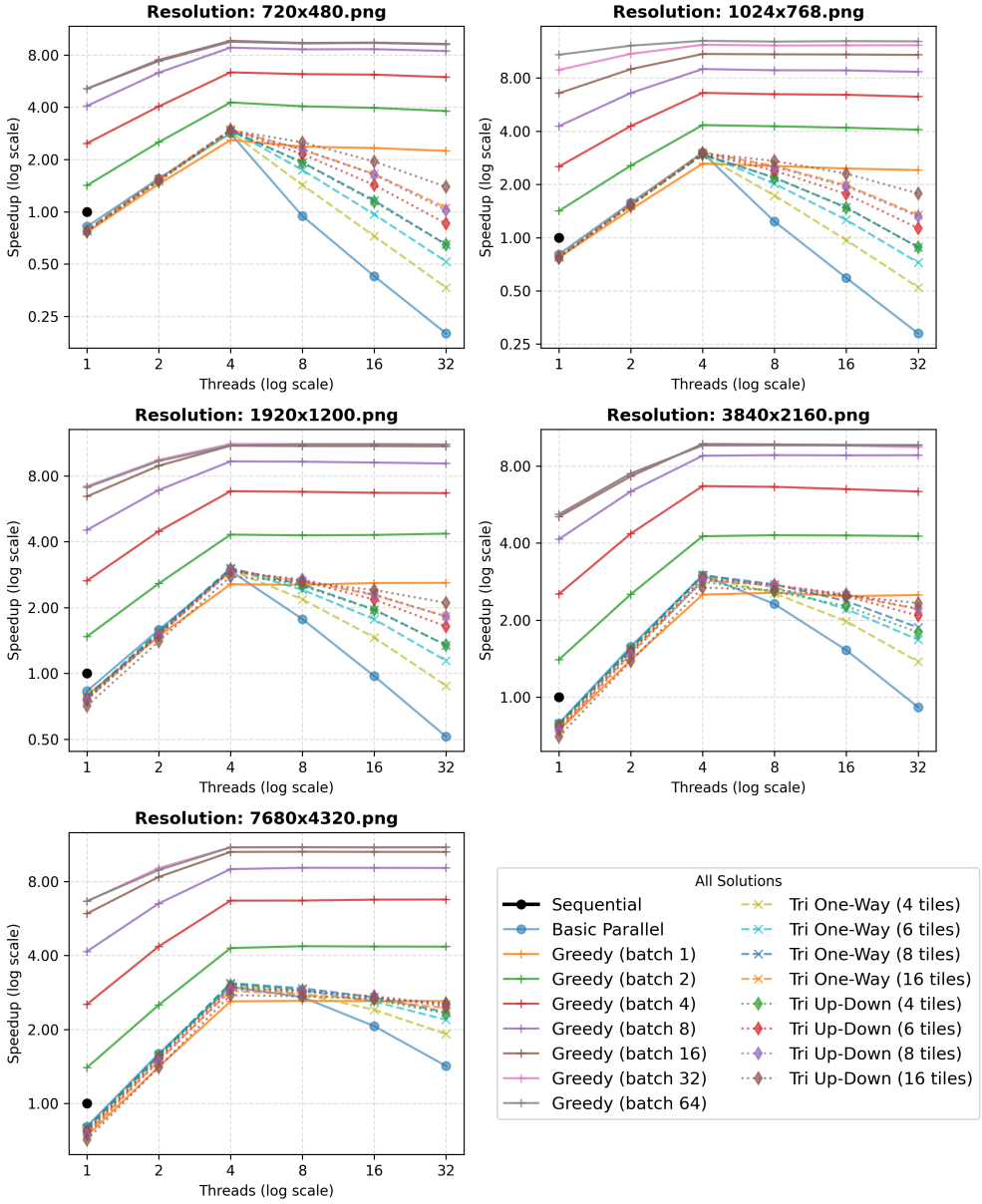

Figure 1: Speedup analysis for various parallel strategies across five image resolutions. Case use of 4 cores.

The following conclusions are drawn:

(a) The optimal performance when using no multi-threading is achieved when the number of threads match the number of cores. In such case each core is exactly working on a thread all the time, hence there is no waiting list and performance is maximum. More threads would cause oversubscription. In figure 1 this behaviour is observed in all parallel tasks and results are qualitatively similar when using 2, 8 and 16 cores.

(b) There is an approximately linear relationship between the number of threads and the achieved speedup, provided the thread count does not exceed the number of physical cores. The parallel efficiency is slightly higher for larger images because the ratio of computation to synchronization overhead improves, and hence there is a more uniform distribution of tasks.

(c) Greddy seam removal performs better than other parallel strategies because we are by-passing several cumulative energy recalculations. Note that performing greedy seam removal is an approximation of the algorithm that may fail. A documented example of this trade-off is provided by Ngor et al., where GPU acceleration dramatically reduces execution time at the expense of global optimality.

(d) The performance of the triangular approach and the basic row-parallel method remains comparable when the thread count is less than or equal to the core count. In this regime, both strategies distribute an equivalent pixel-processing load per thread, and the hardware can accommodate them properly. However, once the number of threads exceeds the number of physical cores, the triangular approach becomes a superior alternative because it present a reduced frequency of thread re-scheduling. When threads exceed cores, the threads compete to get allocated in the CPU, increasing the computation time.

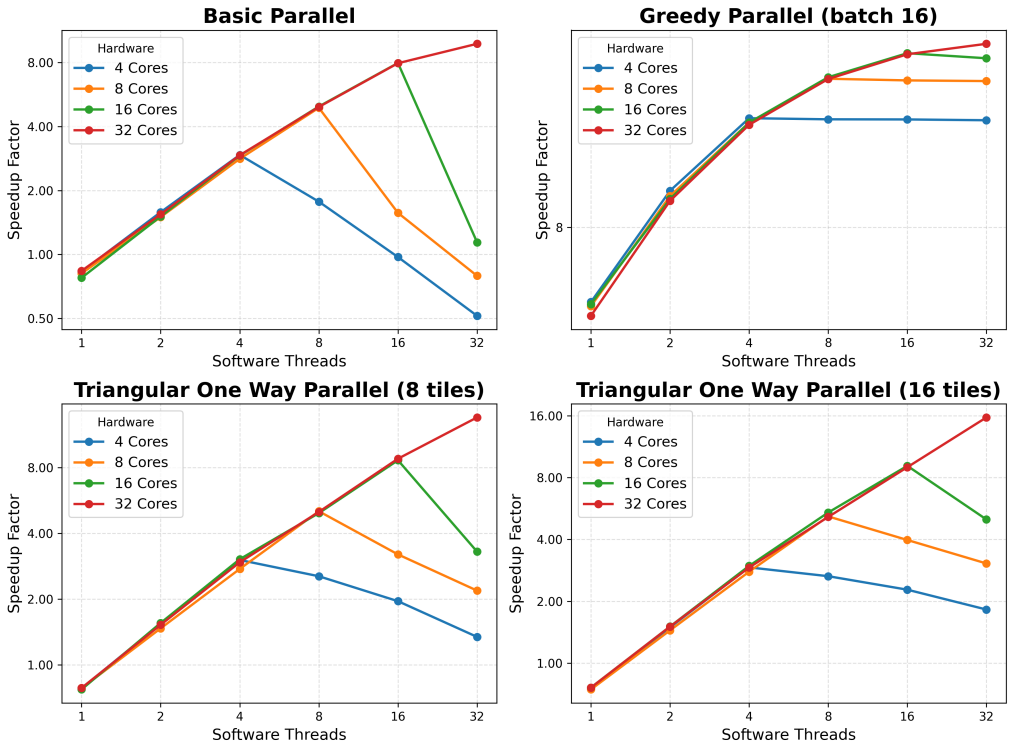

Figure 2: Hardware scalability comparison for the 1920x1200 resolution. Each subplot represents a specific parallel strategy, comparing the speedup achieved across different physical core counts (1, 2, 4, and 8 cores) as the number of threads increases.

(a) Across all parallel implementations, we observe that increasing the thread count yields a corresponding increase in speedup within the undersubscribed regime (where threads $\leq$ cores). However, once the number of threads surpasses the physical core count, performance degrades due to oversubscription. Consequently, the number of available physical cores represents the practical upper bound for the algorithm's performance.

Exercise 2: (a) We compare standard seam removal and greedy seam removal across different images and different number of seams to remove. We observe that the algorithm is robust in the dataset, and doesn't present major differences for even seams 500 seams removed. As shown in figure 3:

(a) (b)

Figure 3: left image using optimized parallel and right image using greedy seam removal.

(b) We need present the heuristics found for the number of threads and the number of cores as a function of the image size, and we also provide an heuristic for the height of the tiles.

- Thread Allocation heuristic: For high-resolution images (1024×768 pixels and above), the optimal thread count is $T = C$, as the high computational density justifies full hardware utilization, shown in Figure 1. For low-resolution images, $T \ll C$ to avoid the disproportionate cost of thread lifecycle management. Hence we define the heuristic:

$$T(W,H,C) = \min\left(C, \left\lfloor \frac{W \times H}{K} C \right\rfloor\right) \quad K \in \{0, 10^4\} \tag{1}$$

- Tile Height Heuristic: In the triangular/wavefront approach, the tile height should be tuned to fit the L2 cache size. This ensures that image computation is large enough to keep all cores busy while remaining small enough to minimize the latency of data storage[1].

$$h_{tile} \approx \frac{\text{L2 Cache Size (bytes)}}{W \times \text{Bytes per Pixel}} \tag{2}$$

(c) To optimize the seam selection process, we modified the aforementioned traditional dynamic programming approach by implementing a bidirectional energy accumulation. The image is bisected at $\bar{y} = \lceil h/2 \rceil$, we execute the parallel triangular approach in parallel, on starting on the top and the other on the bottom as shown in image 8. The final

[1] We observe that an alternative heuristic used is $\sqrt{H}$, although we could not verify empirically the validity of it.

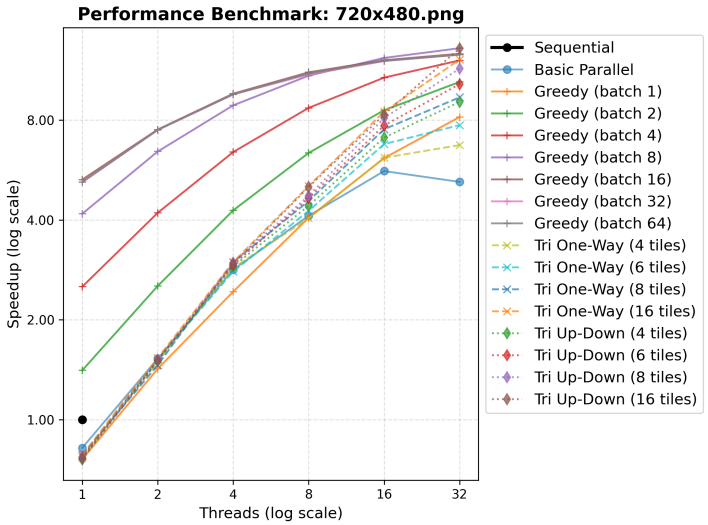

Figure 4: Performance for a first image on a 32 threads.

seam is reconstructed by identifying the optimal midpoint $\hat{x}$ via the formula

$$\hat{x} = \underset{i}{\operatorname{argmin}} \left(E(x_i, \bar{y}) + \min \left(E(x_{i-1}, \bar{y}), E(x_i, \bar{y}), E(x_{i+1}, \bar{y}) \right) \right) \tag{3}$$

and performing two simultaneous tracebacks, one moving upward and one moving downward from the select $\hat{x}$.

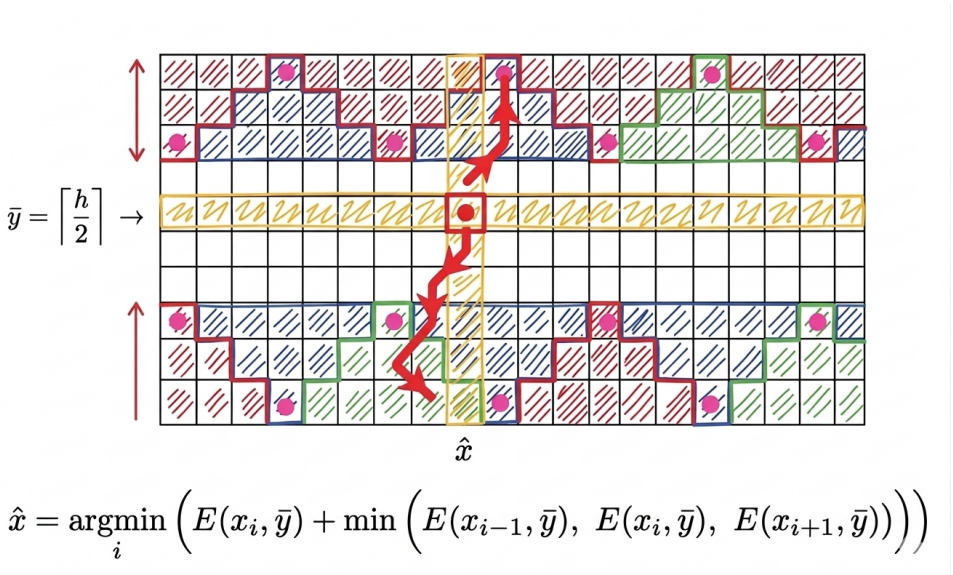

Figure 5: Bidirectional seam carving synchronization at the image midpoint $\bar{y}$. The upward and downward wavefronts (red arrows) converge at the central row, where the optimal transition point $\hat{x}$ is determined by minimizing the combined energy from both computed halves.

As shown in Figure 1, the performance of this approach has proven effective.

1 Comments

(a) We proceed now to implement triangle-based cumulative energy approach. We compared two tiling strategies:

i. **Case 1 ($Y \pm 1$):** The tiles have a horizontal tip of two blocks. This approach present the pink dots not uniformly distributed, see Figure 6a).

ii. **Case 2 ($Y \pm 2$):** The tiles have a single-block tip. This approach present the pink dots uniformly distributed, see Figure 6b).

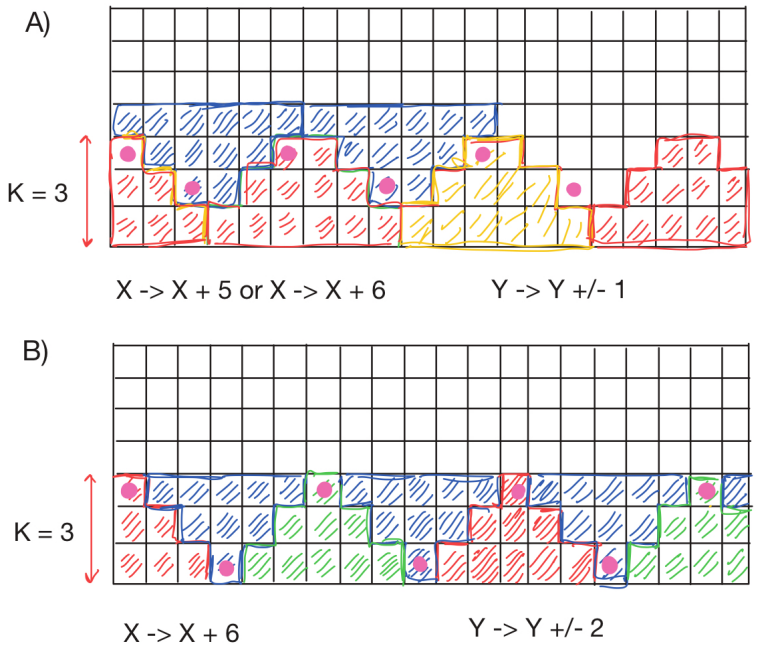

Figure 6: Comparison of wavefront tiling strategies for parallel seam carving for a tile height of 3.

Computationally, both options present similar performance. However, in this study the second approach (single-block tip) seems more suitable because when the height of the triangle is $k = 1$, we fully return to the standard sequential approach, and, hence, we have a robust baseline to compare.

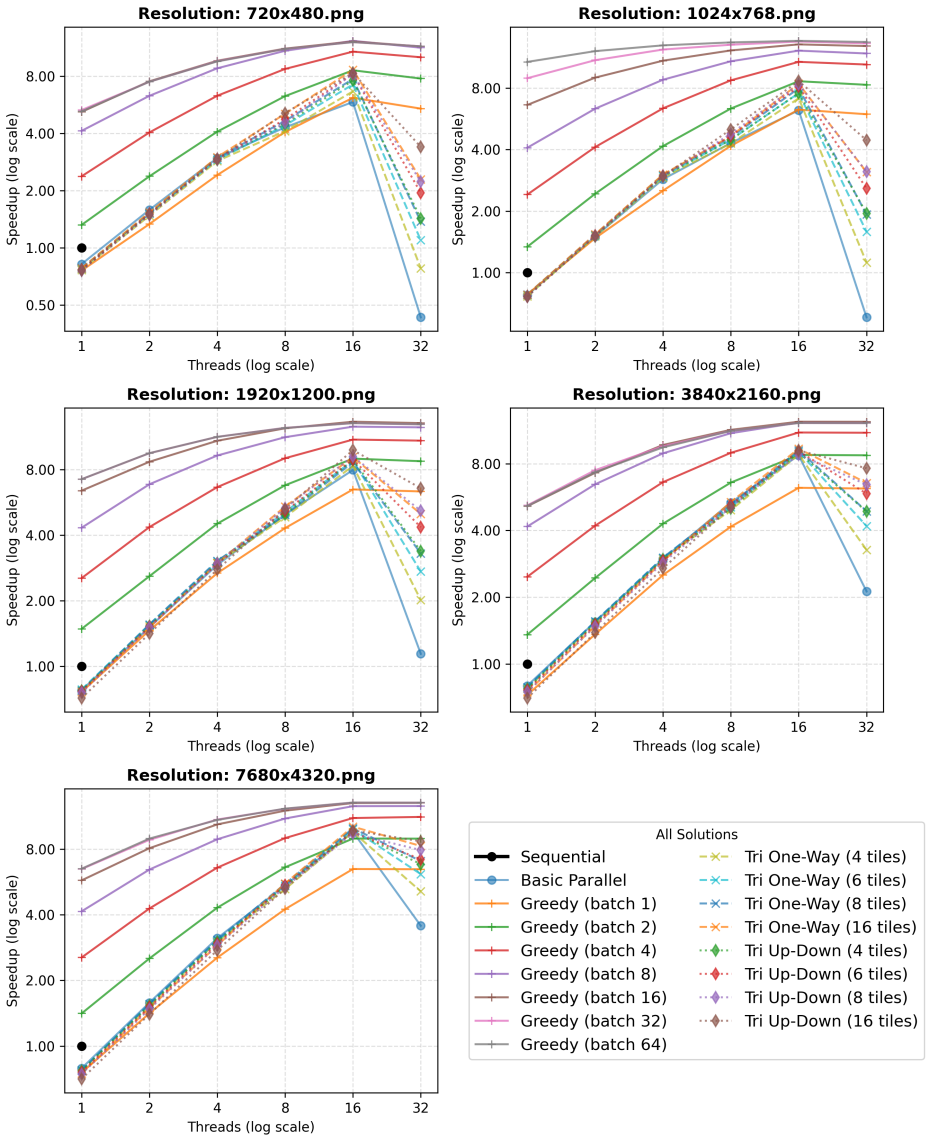

Figure 7: Speedup analysis for various parallel strategies across five image resolutions. Case use of 16 cores.

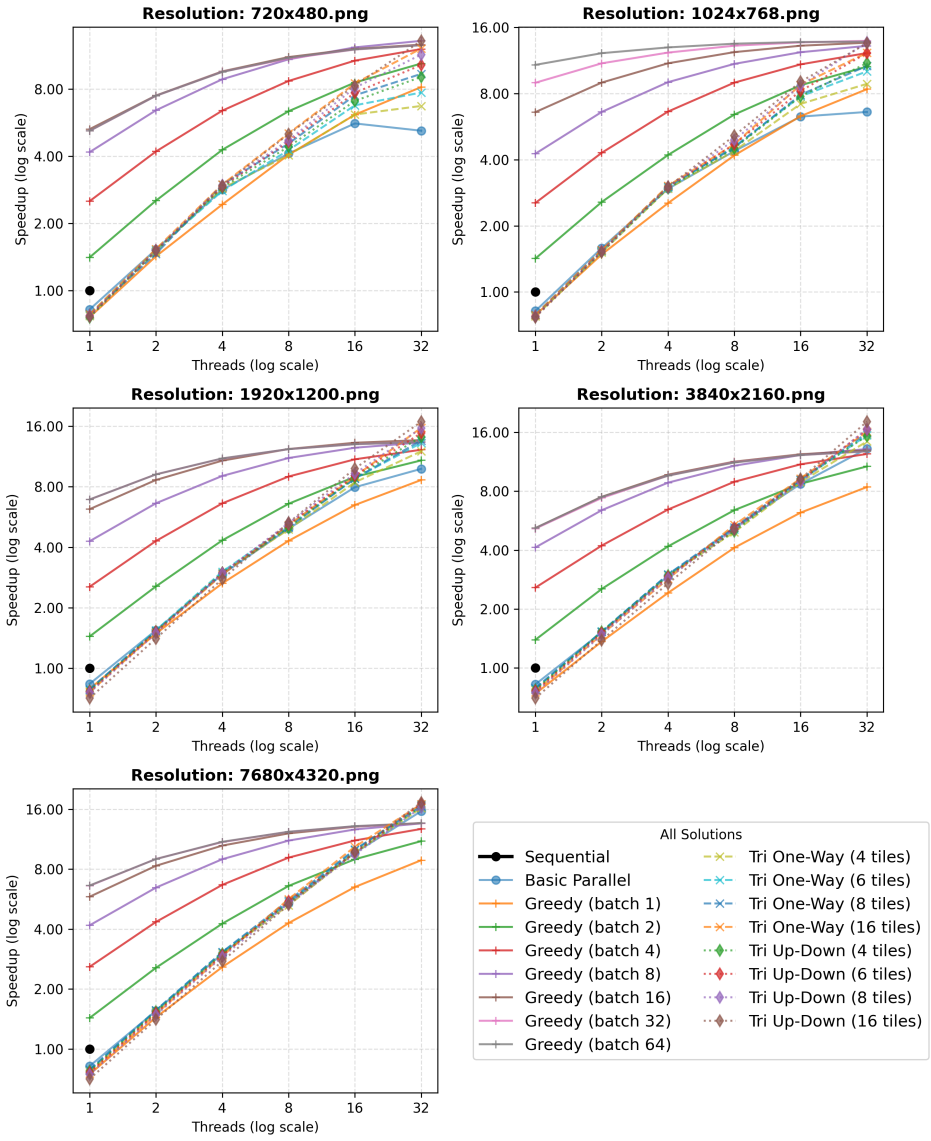

Figure 8: Speedup analysis for various parallel strategies across five image resolutions. Case use of 32 cores.